ASSESSMENT - 1

NAME: M. Rumana khan

REG NO: 192471025

COURSE CODE / NAME: CSA0610 - Design and Analysis Of Algorithms

---

Assignment Title: Scalability Audit of an E-Commerce Platform (“ShopSphere”) – IT Application-Based Problem Solving

---

Introduction

ShopSphere’s product catalogue currently contains n=100,000n=100,000n=100,000 products and is expected to grow to n=1,000,000n=1,000,000n=1,000,000 within a year. Before this expansion, five backend modules (M1 to M5), along with two product-search strategies (S1 and S2), must be analysed to understand how their performance changes as the input size increases.

The aim of this assignment is to identify the basic operation of each module, calculate operation counts for iterative algorithms, solve recurrences for recursive algorithms using backward substitution, the Master Theorem and recursion trees, analyse best, average and worst cases, prove an asymptotic bound, and determine which modules can complete within the nightly processing window.

Conventions used throughout:

- nnn represents the number of products, requests or records, depending on the module.
- All logarithms are base 2.
- The server executes 10 basic operations per second, as specified in the assignment.
- The processing window is 10 minutes, equivalent to 600 seconds.
- Therefore, the maximum number of operations allowed within the window is:

10×600=6,000 operations10 \times 600 = 6,000 \text{ operations}10×600=6,000 operations

The analysis uses the exact operation counts and formulas verified by the Python program in the appendix.

Task (a) – Understand the Problem

For every module, I identify the input size, basic operation, location of the operation in the code, and expected growth of its operation count.

Module	Input size	Basic operation	Expected growth
M1 – Duplicate detector	Number of products	Comparing pairs of products	Quadratic, $\Theta(n^2)$
M2 – API rate limiter	Number of requests	Executing Audit(Request[j])	Linear, $\Theta(n)$
S1 – Sequential search	Number of products	Comparing the search key with a product	Linear, $\Theta(n)$
S2 – Binary search	Size of sorted index	Comparing the key with the middle entry	Logarithmic, $\Theta(\log n)$
M4 – Recommendation engine	Size of customer-product data	Linear merge work at each recursive call	Current: $\Theta(n^2)$; redesigned: $\Theta(n \log n)$
M5 – Sales analytics	Number of sales records	Sorting work at each recursive call	$\Theta(n \log^2 n)$
Report generator	Number of sales records	Dominant quadratic operation	$\Theta(n^2)$

Justification

- M1: Every product is compared with the products that follow it. The number of comparisons grows quadratically.
- M2: The inner loop runs for iii iterations while iii doubles after each outer-loop pass. The total is a geometric series, giving linear growth.
- S1: Products are checked one by one until the key is found or the list ends.
- S2: The search range is halved after every comparison, giving logarithmic growth.
- M4: The current design makes four recursive calls on half-sized inputs, whereas the redesign makes two.
- M5: Each recursive level performs nlognn\log nnlogn work across the corresponding subproblem sizes, resulting in Θ(nlog2n)\Theta(n\log^2 n)Θ(nlog2n).
- Report generator: The quadratic term 3n23n^23n2 dominates the other terms for large inputs.

In summary, M1, the current M4 design and the report generator have quadratic growth. M2 and S1 have linear growth, S2 has logarithmic growth, the redesigned M4 has nlognn\log nnlogn growth, and M5 has nlog2nn\log^2 nnlog2n growth.

Task (b) – Analyse the Iterative Modules

Module M1 – Duplicate-Listing Detector

Basic operation: Comparing the title and seller of two products.

The outer loop selects a product, and the inner loop compares it with every product that follows it.

For a fixed iii, the inner loop performs n−i−1n-i-1n−i−1 comparisons.

Therefore,

C(n)=(n−1)+(n−2)+⋯+2+1C(n)=(n-1)+(n-2)+\cdots+2+1C(n)=(n−1)+(n−2)+⋯+2+1

Using the sum of the first n−1n-1n−1 natural numbers,

C(n)=n(n−1)2C(n)=\frac{n(n-1)}{2}C(n)=2n(n−1)

Expanding,

C(n)=n22−n2C(n)=\frac{n^2}{2}-\frac{n}{2}C(n)=2n2−2n

Since the code has no early exit, the number of comparisons is the same for every input.

- Best case: n(n−1)2\frac{n(n-1)}{2}2n(n−1)
- Average case: n(n−1)2\frac{n(n-1)}{2}2n(n−1)
- Worst case: n(n−1)2\frac{n(n-1)}{2}2n(n−1)
- Time complexity: Θ(n2)\Theta(n^2)Θ(n2)
- Auxiliary space complexity: Θ(1)\Theta(1)Θ(1)

Verification

Number of products (n)	Comparisons n(n − 1)/2
1,000	499,500
100,000	4,999,950,000
1,000,000	499,999,500,000

The Python program confirms these values.

Meaning: Increasing the catalogue size from 100,000 to 1,000,000 products increases the number of comparisons by approximately 100 times. This makes the current duplicate-detection approach unsuitable for large catalogues.

Module M2 – API Rate Limiter

Basic operation: Executing `Audit(Request[j])`.

The outer loop begins with i=1i=1i=1, and iii doubles after every pass:

i=1,2,4,8,…i=1,2,4,8,\ldotsi=1,2,4,8,…

The inner loop executes iii times during each pass.

Therefore, the total number of audits is:

A(n)=1+2+4+⋯+2k−1A(n)=1+2+4+\cdots+2^{k-1}A(n)=1+2+4+⋯+2k−1

Using the geometric-series formula,

A(n)=2k−1A(n)=2^k-1A(n)=2k−1

where k=⌈log2n⌉k=\lceil\log_2 n\rceilk=⌈log2n⌉. Hence,

A(n)=2⌈log2n⌉−1\boxed{A(n)=2^{\lceil\log_2 n\rceil}-1}A(n)=2⌈log2n⌉−1

When nnn is a power of 2, the formula simplifies to:

A(n)=n−1A(n)=n-1A(n)=n−1

For general nnn, the count remains bounded by a constant multiple of nnn. Therefore,

A(n)=Θ(n)\boxed{A(n)=\Theta(n)}A(n)=Θ(n)

Is the claim of Θ(nlogn)\Theta(n\log n)Θ(nlogn) correct?

No. Although the outer loop has logarithmically many passes, the inner-loop sizes are 1,2,4,8,…1,2,4,8,\ldots1,2,4,8,…, not nnn during every pass. Their total forms a geometric series, so the tight bound is Θ(n)\Theta(n)Θ(n).

- Best case: Same operation count for the given input size.
- Average case: Same operation count for the given input size.
- Worst case: Same operation count for the given input size.
- Time complexity: Θ(n)\Theta(n)Θ(n)
- Auxiliary space complexity: Θ(1)\Theta(1)Θ(1)

Verification for n=1,024n=1,024n=1,024

The outer loop runs with i=1,2,4,…,512i=1,2,4,\ldots,512i=1,2,4,…,512, giving 10 passes.

A(1024)=1+2+4+⋯+512=1023A(1024)=1+2+4+\cdots+512=1023A(1024)=1+2+4+⋯+512=1023

The Python output confirms 1,023 audits and 10 outer-loop passes.

n	Audits	Outer-loop passes
1,024	1,023	10
100,000	131,071	17
1,000,000	1,048,575	20

In summary, M1 has quadratic complexity, while M2 has linear complexity. However, under the specified server speed of 10 operations per second, both exceed the 10-minute processing window at n=1,000,000n=1,000,000n=1,000,000.

Task (c) – Solve a Recurrence by Substitution (Search Version S2)

The comparison count of binary search is represented by:

T(n)=T(n/2)+1,T(1)=1T(n)=T(n/2)+1,\qquad T(1)=1T(n)=T(n/2)+1,T(1)=1

Assume n=2kn=2^kn=2k, so the input can be divided exactly in half at every step.

Backward substitution

T(n)=T(n/2)+1T(n)=T(n/2)+1T(n)=T(n/2)+1

=T(n/4)+2=T(n/4)+2=T(n/4)+2

=T(n/8)+3=T(n/8)+3=T(n/8)+3

Continuing this pattern,

T(n)=T(n/2i)+iT(n)=T(n/2^i)+iT(n)=T(n/2i)+i

The recursion reaches the base case when:

n2i=1\frac{n}{2^i}=12in=1

Therefore,

i=log2ni=\log_2 ni=log2n

Substituting into the recurrence,

T(n)=T(1)+log2nT(n)=T(1)+\log_2 nT(n)=T(1)+log2n

Since T(1)=1T(1)=1T(1)=1,

T(n)=1+log2n\boxed{T(n)=1+\log_2 n}T(n)=1+log2n

For input sizes that are not powers of 2, the corresponding count is:

T(n)=⌊log2n⌋+1\boxed{T(n)=\lfloor\log_2 n\rfloor+1}T(n)=⌊log2n⌋+1

Verification

For n=1,024n=1,024n=1,024,

T(1024)=log21024+1=10+1=11T(1024)=\log_2 1024+1=10+1=11T(1024)=log21024+1=10+1=11

For n=1,000,000n=1,000,000n=1,000,000,

T(n)=⌊19.93⌋+1=20T(n)=\lfloor19.93\rfloor+1=20T(n)=⌊19.93⌋+1=20

The Python output confirms both values.

Complexity analysis

Implementation	Time complexity	Auxiliary space complexity
Iterative binary search	Θ(log n)	Θ(1)
Recursive binary search	Θ(log n)	Θ(log n)

Meaning: Doubling the input size adds approximately one additional comparison. At n=1,000,000n=1,000,000n=1,000,000, binary search needs only 20 comparisons in the worst case.

Conclusion: S2 is the most scalable search strategy among the two search methods considered. At the assumed server speed, its worst-case processing time is:

2010=2 seconds\frac{20}{10}=2\text{ seconds}1020=2 seconds

Therefore, S2 passes the 10-minute window.

Task (d) – Apply the Master Theorem

The Master Theorem analyses recurrences of the form:

T(n)=aT(n/b)+f(n)T(n)=aT(n/b)+f(n)T(n)=aT(n/b)+f(n)

We compare f(n)f(n)f(n) with nlogban^{\log_b a}nlogba to determine the asymptotic complexity.

M4 – Recommendation Engine

The current design uses:

T(n)=4T(n/2)+nT(n)=4T(n/2)+nT(n)=4T(n/2)+n

The proposed redesign uses:

T(n)=2T(n/2)+nT(n)=2T(n/2)+nT(n)=2T(n/2)+n

Comparison of the two designs

Parameter	Current design	Proposed redesign
Recurrence	T(n) = 4T(n/2) + n	T(n) = 2T(n/2) + n
a	4	2
b	2	2
f(n)	n	n
n^(log_b a)	n²	n
Master Theorem case	Case 1	Case 2
Time complexity	Θ(n²)	Θ(n log n)

Current design

For the current recurrence,

T(n)=4T(n/2)+nT(n)=4T(n/2)+nT(n)=4T(n/2)+n

Here, a=4a=4a=4, b=2b=2b=2, and f(n)=nf(n)=nf(n)=n.

nlog24=n2n^{\log_2 4}=n^2nlog24=n2

Since f(n)=nf(n)=nf(n)=n is polynomially smaller than n2n^2n2, Master Theorem Case 1 applies.

T(n)=Θ(n2)\boxed{T(n)=\Theta(n^2)}T(n)=Θ(n2)

For powers of 2 and the base case T(1)=1T(1)=1T(1)=1, the exact solution is:

T(n)=2n2−n\boxed{T(n)=2n^2-n}T(n)=2n2−n

Proposed redesign

The redesigned recurrence is:

T(n)=2T(n/2)+nT(n)=2T(n/2)+nT(n)=2T(n/2)+n

Here, a=2a=2a=2 and b=2b=2b=2.

nlog22=nn^{\log_2 2}=nnlog22=n

Since f(n)=nf(n)=nf(n)=n has the same order as nlogban^{\log_b a}nlogba, Master Theorem Case 2 applies.

T(n)=Θ(nlogn)\boxed{T(n)=\Theta(n\log n)}T(n)=Θ(nlogn)

For powers of 2 and T(1)=1T(1)=1T(1)=1, the exact solution is:

T(n)=nlog2n+n\boxed{T(n)=n\log_2 n+n}T(n)=nlog2n+n

Comparison of the designs

The current design performs four recursive calls on half-sized inputs, while the redesign performs only two. This reduces the asymptotic growth from quadratic to nlognn\log nnlogn.

For n=1,024n=1,024n=1,024, the Python output confirms:

- Current design: 2,096,1282,096,1282,096,128 operations
- Redesigned design: 11,26411,26411,264 operations

For n=1,000,000n=1,000,000n=1,000,000, the formula-based counts are:

- Current design: 1,999,999,000,0001,999,999,000,0001,999,999,000,000 operations
- Redesigned design: approximately 20,931,568.5720,931,568.5720,931,568.57 operations

Although the redesign reduces the operation count considerably, it still exceeds the available 6,000-operation budget at one million products.

M5 – Sales Analytics Engine

The recurrence is:

T(n)=2T(n/2)+nlognT(n)=2T(n/2)+n\log nT(n)=2T(n/2)+nlogn

Here,

a=2,b=2,f(n)=nlogna=2,\quad b=2,\quad f(n)=n\log na=2,b=2,f(n)=nlogn

Therefore,

nlogba=nlog22=nn^{\log_b a}=n^{\log_2 2}=nnlogba=nlog22=n

The function nlognn\log nnlogn is larger than nnn, but it is not polynomially larger than nnn. Consequently, none of the three basic Master Theorem cases applies directly.

Using the extended Master Theorem, if

f(n)=Θ(nlogbalogkn)f(n)=\Theta(n^{\log_b a}\log^k n)f(n)=Θ(nlogbalogkn)

then

T(n)=Θ(nlogbalogk+1n)T(n)=\Theta(n^{\log_b a}\log^{k+1}n)T(n)=Θ(nlogbalogk+1n)

Here, k=1k=1k=1. Therefore,

T(n)=Θ(nlog2n)\boxed{T(n)=\Theta(n\log^2 n)}T(n)=Θ(nlog2n)

Confirmation using a recursion tree

Let L=log2nL=\log_2 nL=log2n.

At level iii:

- Number of nodes: 2i2^i2i
- Size of each subproblem: n/2in/2^in/2i
- Work per node: (n/2i)log2(n/2i)(n/2^i)\log_2(n/2^i)(n/2i)log2(n/2i)

Therefore, the total work at level iii is:

2i(n2i)(L−i)=n(L−i)2^i\left(\frac{n}{2^i}\right)(L-i)=n(L-i)2i(2in)(L−i)=n(L−i)

Summing the work over the internal levels and adding the leaves gives:

T(n)=n∑i=0L−1(L−i)+nT(n)=n\sum_{i=0}^{L-1}(L-i)+nT(n)=ni=0∑L−1(L−i)+n

T(n)=nL(L+1)2+nT(n)=n\frac{L(L+1)}{2}+nT(n)=n2L(L+1)+n

Hence,

T(n)=nlog2n(log2n+1)2+n\boxed{T(n)=\frac{n\log_2 n(\log_2 n+1)}{2}+n}T(n)=2nlog2n(log2n+1)+n

For n=1,024n=1,024n=1,024, where L=10L=10L=10,

T(1024)=102410(11)2+1024=57,344T(1024)=1024\frac{10(11)}{2}+1024=57,344 T(1024)=1024210(11)+1024=57,344

This matches the Python output.

For n=1,000,000n=1,000,000n=1,000,000, the formula gives approximately 209,599,497.10209,599,497.10209,599,497.10 operations. This is much smaller than the current M4 design, but it still exceeds the 6,000-operation budget.

Conclusion: The redesigned M4 is Θ(nlogn)\Theta(n\log n)Θ(nlogn), while M5 is Θ(nlog2n)\Theta(n\log^2 n)Θ(nlog2n). Both are improvements over a quadratic design, but neither meets the specified 10-minute processing limit at one million records.

Task (e) – Best, Average and Worst Cases of S1 and S2

S1 – Sequential Search of the Unsorted Catalogue

Sequential search examines products one by one until it finds the required product or reaches the end of the catalogue.

Best case: The product is the first entry.

Cbest(n)=1=Θ(1)C_{\text{best}}(n)=1=\Theta(1)Cbest(n)=1=Θ(1)

Worst case: The product is the last entry or is absent.

Cworst(n)=n=Θ(n)C_{\text{worst}}(n)=n=\Theta(n)Cworst(n)=n=Θ(n)

Average case when the product exists: Assuming the product is equally likely to appear at any position,

Cavg(n)=1+2+⋯+nnC_{\text{avg}}(n)=\frac{1+2+\cdots+n}{n}Cavg(n)=n1+2+⋯+n

Cavg(n)=n+12\boxed{C_{\text{avg}}(n)=\frac{n+1}{2}}Cavg(n)=2n+1

If a fraction ppp of searches succeed and the rest fail, the average is:

Cavg(n)=pn+12+(1−p)n\boxed{C_{\text{avg}}(n)=p\frac{n+1}{2}+(1-p)n}Cavg(n)= p2n+1+(1−p)n

For n=1,000,000n=1,000,000n=1,000,000 and p=0.5p=0.5p=0.5,

Cavg(n)=0.5(1,000,0012)+0.5(1,000,000)C_{\text{avg}}(n)=0.5\left(\frac{1,000,00 1}{2}\right)+0.5(1,000,000)Cavg(n)=0.5(21,000,001)+0.5(1,000,000)

Cavg(n)=750,000.25\boxed{C_{\text{avg}}(n)=750,000.25}Cavg(n)=750,000.25

S2 – Binary Search of the Sorted Index

Binary search compares the key with the middle entry and discards half of the remaining search range after each comparison.

Best case: The key is found in the first comparison.

Cbest(n)=1=Θ(1)C_{\text{best}}(n)=1=\Theta(1)Cbest(n)=1=Θ(1)

Worst case: The key is absent or found at the deepest level.

Cworst(n)=⌊log2n⌋+1\boxed{C_{\text{worst}}(n)=\lfloor\log_2 n\rfloor+1}Cworst(n)=⌊log2n⌋+1

For n=1,000,000n=1,000,000n=1,000,000, the worst-case count is 20 comparisons.

Average case: For a successful search, the average depends on the distribution of keys in the binary-search tree. For a large, approximately balanced tree, it can

be estimated as roughly log2n−1\log_2 n-1log2n−1, or approximately 18.93 comparisons at one million records.

Using p=0.5p=0.5p=0.5 and an unsuccessful search count of 20:

Cavg≈0.5(18.93)+0.5(20)C_{\text{avg}}\approx0.5(18.93)+0.5(20)Cavg≈0.5(18.93)+0.5(20)

Cavg≈19.47\boxed{C_{\text{avg}}\approx19.47}Cavg≈19.47

This is an approximation rather than an exact average for every possible input size and key distribution.

Comparison at n = 1,000,000

Case	S1 – Sequential search	S2 – Binary search
Best	1 comparison	1 comparison
Average, p = 0.5	750,000.25 comparisons	Approximately 19.47 comparisons
Worst	1,000,000 comparisons	20 comparisons
Time complexity	Θ(n)	Θ(log n)

Meaning: Binary search is substantially more efficient for large catalogues, provided the data is sorted. However, the cost of maintaining or preparing the sorted index should also be considered.

Conclusion: S2 is the preferred search strategy for ShopSphere when the sorted index can be maintained efficiently. Its 20 worst-case comparisons take approximately 2 seconds at the specified server speed, so it passes the processing window.

Task (f) – Prove an Asymptotic Bound

The report generator has the function:

f(n)=3n2+10nlog2n+50f(n)=3n^2+10n\log_2 n+50f(n)=3n2+10nlog2n+50

The objective is to prove that:

f(n)=Θ(n2)\boxed{f(n)=\Theta(n^2)}f(n)=Θ(n2)

By definition, we must find positive constants c1,c2c_1,c_2c1,c2 and a value n0n_0n0 such that:

c1n2≤f(n)≤c2n2c_1n^2\leq f(n)\leq c_2n^2c1n2≤f(n)≤c2n2

for every n≥n0n\geq n_0n≥n0.

Lower bound

For n≥1n\geq1n≥1, both 10nlog2n10n\log_2 n10nlog2n and 50 are non-negative.

Therefore,

f(n)=3n2+10nlog2n+50≥3n2f(n)=3n^2+10n\log_2 n+50\geq3n^2f(n)=3n2+10nlog2n+50≥3n2

Thus, the lower-bound constants can be chosen as:

c1=3c_1=3c1=3

This proves:

f(n)=Ω(n2)f(n)=\Omega(n^2)f(n)=Ω(n2)

Upper bound

For n≥1n\geq1n≥1,

log2n≤n\log_2 n\leq nlog2n≤n

Therefore,

10nlog2n≤10n210n\log_2 n\leq10n^210nlog2n≤10n2

Also,

50≤50n250\leq50n^250≤50n2

Combining these inequalities,

f(n)≤3n2+10n2+50n2f(n)\leq3n^2+10n^2+50n^2f(n)≤3n2+10n2+50n2

f(n)≤63n2f(n)\leq63n^2f(n)≤63n2

Thus, the upper-bound constant can be chosen as:

c2=63c_2=63c2=63

This proves:

f(n)=O(n2)f(n)=O(n^2)f(n)=O(n2)

Final result

Choosing c1=3c_1=3c1=3, c2=63c_2=63c2=63, and n0=1n_0=1n0=1, we obtain:

3n2≤f(n)≤63n2\boxed{3n^2\leq f(n)\leq63n^2}3n2≤f(n)≤63n2

Therefore,

f(n)=Θ(n2)\boxed{f(n)=\Theta(n^2)}f(n)=Θ(n2)

Numerical verification

n	f(n)	$3n^2$ (lower bound)	$63n^2$ (upper bound)	Bound holds?
1	53.00	3	63	Yes
10	682.19	300	6,300	Yes
100	36,693.86	30,000	630,000	Yes
1,000	3,099,707.84	3,000,000	63,000,000	Yes
1,000,000	3,000,199,315,735.69	3,000,000,000,000	63,000,000,000,000	Yes

The Python output confirms that the bound holds for all the tested values.

Proving that f(n)f(n)f(n) is not O(n)O(n)O(n)

Suppose, for contradiction, that f(n)=O(n)f(n)=O(n)f(n)=O(n). Then there must be a constant c>0c>0c>0 such that:

f(n)≤cnf(n)\leq cnf(n)≤cn

for sufficiently large nnn.

However, since f(n)≥3n2f(n)\geq3n^2f(n)≥3n2,

3n2≤cn3n^2\leq cn3n2≤cn

Dividing by n>0n>0n>0,

3n≤c3n\leq c3n≤c

This cannot hold for every sufficiently large nnn, because nnn can grow without bound.

Therefore,

f(n)≠O(n)\boxed{f(n)\neq O(n)}f(n)=O(n)

Conclusion: The report generator has quadratic time complexity. Its quadratic term dominates the other terms for large inputs, making it unsuitable for the specified processing window at one million records.

Task (g) – Scalability Decision

The final scalability analysis uses the verified operation counts at n=1,000,000n=1,000,000n=1,000,000.

The server performs 10 basic operations per second, and the nightly processing window is 600 seconds.

Therefore,

Running time=Operation count10\text{Running time}=\frac{\text{Operation count}}{10}Running time=10Operation count

The maximum permitted operation count is:

10×600=6,00010\times600=6,00010×600=6,000

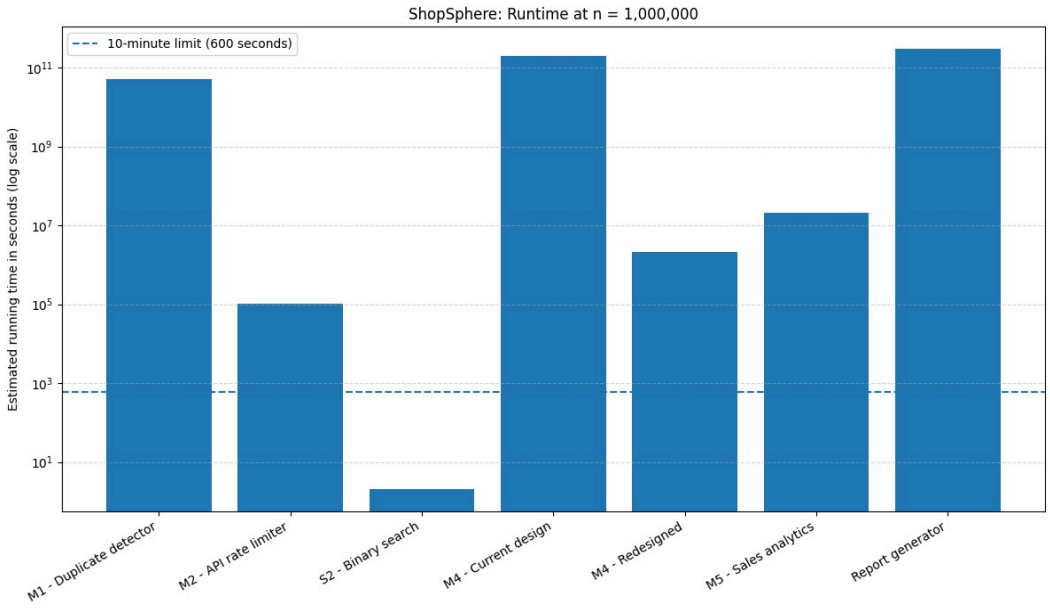
Any module requiring more than 6,000 basic operations fails the 10-minute processing window under the stated assumption.

Final scalability table

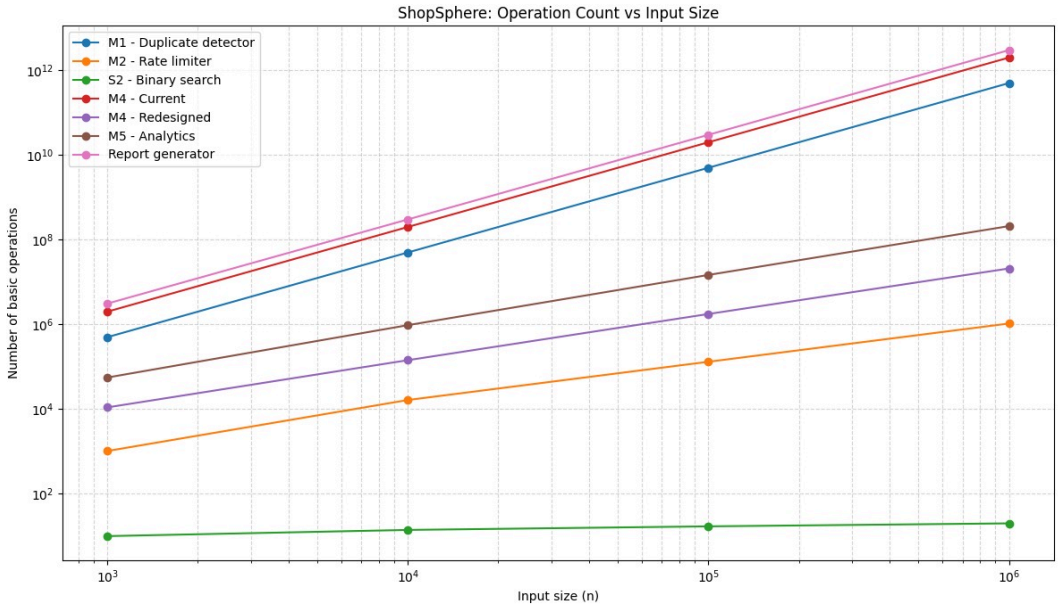
Module	Operations at n = 1,000,000	Running time at 10 ops/s	Within 10 minutes?
M1 – Duplicate detector	499,999,500,000	49,999,950,000 s	No – fails
M2 – API rate limiter	1,048,575	104,857.5 s	No – fails
S2 – Binary search	20	2 s	Yes
M4 – Current design	1,999,999,000,000	199,999,900,000 s	No – fails
M4 – Redesigned	20,931,568.57	2,093,156.86 s	No – fails
M5 – Sales analytics	209,599,497.10	20,959,949.71 s	No – fails
Report generator	3,000,199,315,735.69	300,019,931,573.57 s	No – fails

Important interpretation: These running times follow the assignment's stated rate of 10 basic operations per second. They are theoretical estimates based on operation counts, not measured wall-clock execution times on an actual server.

Graph 1 – Runtime Comparison at $n = 1,000,000$



Graph 2 – Operations against input size



The graph compares the operation counts of M1, M2, S2, the current and redesigned M4, M5, and the report generator as the input size increases.

The horizontal axis represents input size nnn, and the vertical axis represents the number of basic operations. A logarithmic scale can be used for both axes to make the differences between the algorithms easier to see.

A horizontal reference line at 6,000 operations represents the maximum work allowed within the 10-minute window.

The graph shows that:

- M1, the current M4 design and the report generator grow quadratically.
- M2 grows linearly but still exceeds the operation budget at one million records.
- The redesigned M4 grows as nlognn\log nnlogn, which is a major improvement over the current design, but it still exceeds the budget.
- M5 grows as nlog2nn\log^2 nnlog2n and also exceeds the budget.
- S2 grows logarithmically and remains well below the limit.

Graph conclusion: Only S2 stays within the 6,000-operation budget at n=1,000,000n=1,000,000n=1,000,000. The remaining modules require further optimisation or a change in the processing assumptions.

Modules that fail the 10-minute window

At one million records, all modules except S2 exceed the 6,000-operation limit.

1. M1 – Duplicate detector

The current approach performs approximately 5×10115\times10^{11}5×1011 comparisons. Replace the all-pairs comparison with a hash set keyed by (title, seller).

- Scan each product once.
- Check whether its key is already present in the set.
- If present, mark it as a duplicate; otherwise, add it to the set.

This reduces the expected time complexity from Θ(n2)\Theta(n^2)Θ(n2) to Θ(n)\Theta(n)Θ(n), at the cost of Θ(n)\Theta(n)Θ(n) additional memory.

An alternative is to sort the products by (title, seller) and compare neighbouring entries. This takes Θ(nlogn)\Theta(n\log n)Θ(nlogn) time.

2. M2 – API rate limiter

M2 is theoretically efficient because it has Θ(n)\Theta(n)Θ(n) time complexity. However, at one million records, it performs 1,048,575 audits, which exceeds the available budget.

Possible improvements include reducing the number of audited requests, processing requests incrementally, or distributing the work across multiple processing resources. Any change must preserve the rate limiter's correctness and security requirements.

3. S2 – Binary search

S2 requires only 20 comparisons in the worst case at one million records. Its estimated running time is 2 seconds.

- Keep the index sorted.
- Use binary search for product lookups.
- Update the index when products are added or modified.

S2 passes the processing window and is the preferred search approach when a sorted index is available.

4. M4 – Recommendation engine

The current design has Θ(n2)\Theta(n^2)Θ(n2) complexity, while the redesign has Θ(nlogn)\Theta(n\log n)Θ(nlogn) complexity.

The redesign reduces the operation count substantially, but its estimated count at one million records is still about 20.93 million operations.

- Adopt the redesign because it has a better asymptotic growth rate.
- Consider optimising repeated work, caching results, or processing the recommendation workload incrementally.
- If the 10-minute limit is strict, additional optimisation or parallel processing must be evaluated.

5. M5 – Sales analytics

M5 has Θ(nlog2n)\Theta(n\log^2 n)Θ(nlog2n) complexity and requires approximately 209.6 million operations at one million records.

An optional improvement is to sort once and merge the sorted halves instead of repeating sorting work at every level. If this change is valid for the module's requirements, the recurrence becomes:

T(n)=2T(n/2)+nT(n)=2T(n/2)+nT(n)=2T(n/2)+n

Therefore,

T(n)=Θ(nlogn)T(n)=\Theta(n\log n)T(n)=Θ(nlogn)

This is an asymptotic improvement, but its actual operation count and ability to meet the 6,000-operation limit must be re-evaluated after the redesign.

6. Report generator

The report generator performs approximately 3×10123\times10^{12}3×1012 operations at one million records. Its quadratic term must be investigated.

- Identify the nested loop or repeated scan responsible for the n2n^2n2 work.
- Replace repeated scans with hash lookups or a single-pass aggregation where possible.
- Use sorting and grouping if the output requirements permit it.
- Recalculate the operation count after redesigning the quadratic step.

Overall recommendation

The scalability audit identifies S2 as the only module that currently satisfies the 10-minute processing limit under the specified rate of 10 basic operations per second.

M1, M2, the current and redesigned M4, M5, and the report generator all exceed the limit at one million records. Improving their asymptotic complexity is necessary, but the redesigned algorithms must also be checked against the strict 6,000-operation budget.

Appendix – Python Program Used to Check the Formulas

The following program verifies the operation counts, recurrence solutions, average search count, and asymptotic bounds. Memoization is used for the recursive functions so that larger input sizes can be checked without repeatedly evaluating the same subproblems.

```
import math

from functools import lru_cache



# M1: Duplicate detector

def m1(n):

    return n * (n - 1) // 2



# M2: API rate limiter

def m2(n):

    i = 1

    count = 0

    outer = 0
```

```
    while i < n:
        count += i
        outer += 1
        i *= 2

    return count, outer



# S2: Binary search comparison count
def s2(n):
    return math.floor(math.log2(n)) + 1



# M4 current design
@lru_cache(maxsize=None)
def m4_current(n):
    if n <= 1:
        return 1
    return 4 * m4_current(n // 2) + n
```

```
# M4 redesigned
@lru_cache(maxsize=None)
def m4_redesigned(n):
    if n <= 1:
        return 1
    return 2 * m4_redesigned(n // 2) + n



# M5: Sales analytics
@lru_cache(maxsize=None)
def m5(n):
    if n <= 1:
        return 1
    return 2 * m5(n // 2) + n * math.log2(n)



# Report generator
def report_generator(n):
    return 3 * n**2 + 10 * n * math.log2(n) + 50
```

```
print("=== M1: Duplicate comparisons ===")
for n in (1000, 100000, 1000000):
    print(f"n = {n:,} | Comparisons = {m1(n):,}")



print("\n=== M2: API rate limiter ===")
for n in (1024, 100000, 1000000):
    audits, outer = m2(n)
    print(
        f"n = {n:,} | Audits = {audits:,} "
        f"| Outer passes = {outer}"
    )



print("\n=== S2 and recurrences ===")

# Direct recurrence checks at n = 1024
n = 1024
L = math.log2(n)
```

```
print(f"\nn = {n:,}")
print("S2 recursive count =", s2(n))
print("S2 formula =", math.floor(L) + 1)

print("M4 current =", m4_current(n))
print("M4 current formula =", 2 * n**2 - n)

print("M4 redesigned =", m4_redesigned(n))
print("M4 redesigned formula =", int(n * L + n))

print("M5 =", m5(n))
print("M5 formula =", int(n * L * (L + 1) / 2 + n))



# Formula-based checks for n = 2**20
# This avoids unnecessary direct recursion for very large counts.
n = 2**20
L = math.log2(n)

print(f"\nn = {n:,}")
print("S2 formula =", math.floor(L) + 1)
```

```
print("M4 current formula =", 2 * n**2 - n)

print("M4 redesigned formula =", int(n * L + n))

print("M5 formula =", int(n * L * (L + 1) / 2 + n))



print("\n=== S1 average comparisons ===")

n = 1_000_000

p = 0.5


s1_average = p * ((n + 1) / 2) + (1 - p) * n


print("S1 average =", s1_average)

print("S2 worst case =", s2(n))



print("\n=== Report generator bound checks ===")

for n in (1, 10, 100, 1000, 1000000):

    f = report_generator(n)

    lower = 3 * n**2

    upper = 63 * n**2
```

```
    print(
        f"n={n:,} | f(n)={f:.2f} "
        f"| 3n^2={lower:,} "
        f"| 63n^2={upper:,} "
        f"| Bound holds: {lower <= f <= upper}"
    )



print("\n=== SCALABILITY TABLE ===")

RATE = 10
WINDOW = 600

n = 1_000_000
L = math.log2(n)

modules = [
    ("M1 - Duplicate detector", m1(n)),
    ("M2 - API rate limiter", m2(n)[0]),
    ("S2 - Binary search", s2(n)),
    ("M4 - Current design", 2 * n**2 - n),
```

```
    ("M4 - Redesigned", n * L + n),
    ("M5 - Sales analytics", n * L * (L + 1) / 2 + n),
    ("Report generator", report_generator(n)),
]

print("Assumed rate:", RATE, "operations/second")
print("Processing window:", WINDOW, "seconds")
print("Maximum operations:", RATE * WINDOW)
print()

print(
    f"{'Module':<28}"
    f"{'Operations':>24}"
    f"{'Time (seconds)':>24}"
    f"{'Within 10 min?':>20}"
)

for name, operations in modules:
    time_seconds = operations / RATE
    within_window = time_seconds <= WINDOW
```

```
    print(
        f"{name:<28}"
        f"{operations:>24,.2f}"
        f"{time_seconds:>24,.2f}"
        f"{str(within_window):>20}"
    )
```

Output interpretation

The program verifies that:

- M1 performs n(n−1)/2n(n-1)/2n(n−1)/2 comparisons.
- M2 performs 1,023 audits at n=1,024n=1,024n=1,024, with 10 outer-loop passes.
- S2 performs 11 comparisons at n=1,024n=1,024n=1,024 and 20 comparisons at n=1,000,000n=1,000,000n=1,000,000.
- The recurrence values match the formulas for the tested power-of-two input sizes.
- The report generator satisfies the chosen quadratic lower and upper bounds.
- The scalability table uses the specified rate of 10 operations per second and a 600-second processing window.

The final scalability conclusion is that only S2 passes the 10-minute window at one million records under the assignment's stated assumptions.

Code of Conduct

I, M. Rumana Khan (Reg. No. 192471025), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____M.RUMANA KHAN_____

Date: _____12-10-2026_____